

Explain Interval Counting Code

Let's first understand **what the question is asking**, because both codes are trying to solve the same problem but use very different ideas.

1. What is the question?

You are given a list of intervals:

```
[1, 3]
[2, 5]
[7, 9]
[4, 8]
```

Two intervals are **intersecting/overlapping** if they have at least one common point.

For two intervals:

```
[a, b]
[c, d]
```

they intersect when:

```
a <= d && c <= b
```

or equivalently, if we assume $a \leq c$:

```
b >= c
```

Example

```
[1, 5]
[3, 7]
```

They intersect:

```
1 ---- 5
   3 ---- 7
```

because $5 \geq 3$.

But:

```
[1, 3]
      [5, 7]
```

do not intersect because:

```
3 < 5
```

So the goal is:

Count how many pairs of intervals intersect.

Code 1 — countIntersectingIntervals

Let's understand this one very carefully.

```
long long countIntersectingIntervals(vector<vector<int>>& intervals) {
    int n = intervals.size();

    vector<int> ends;

    for(auto &interval : intervals){
        ends.push_back(interval[1]);
    }

    sort(ends.begin(), ends.end());

    long long total = 0;

    for(auto &interval : intervals){
        int start = interval[0];

        auto it = lower_bound(
```

```
        ends.begin(),
        ends.end(),
        start
    );

    total += ends.end() - it;
}

long long totalp = 1LL * n * (n - 1) / 2;

return total - n - totalp;
}
```

Main idea of Code 1

The important idea is:

Instead of directly comparing every pair of intervals, look at the **end points**.

Suppose:

```
A = [1, 5]
B = [3, 7]
```

For B:

```
start = 3
```

If an interval has:

```
end >= 3
```

then it can intersect B.

Here:

```
A ends at 5
5 >= 3
```

so A intersects B.

Step 1 — Store all ending points

Suppose:

```
intervals =

[1, 5]
[3, 7]
[8, 10]
[6, 9]
```

We extract:

```
ends.push_back(interval[1]);
```

So:

```
ends = [5, 7, 10, 9]
```

Then:

```
sort(ends.begin(), ends.end());
```

becomes:

```
ends = [5, 7, 9, 10]
```

Now we have a sorted list of all interval ending points.

Step 2 — Process every interval's start

Take:

```
[3, 7]
```

Its start is:

```
start = 3
```

We want to know:

```
How many intervals have end >= 3?
```

Our ends are:

```
5 7 9 10
```

All four satisfy:

```
end >= 3
```

So:

```
4 intervals
```

How does `lower_bound` help?

This:

```
auto it = lower_bound(ends.begin(), ends.end(), start);
```

finds the **first element** `>= start`.

For:

```
ends = [5, 7, 9, 10]  
start = 3
```

`lower_bound` points to:

```
5  
↑
```

Therefore:

```
ends.end() - it
```

is:

```
4
```

because there are four elements from 5 to 10.

Why are we counting `end >= start`?

Consider:

```
A = [1, 5]  
B = [3, 7]
```

For B:

```
B.start = 3
```

A ends at:

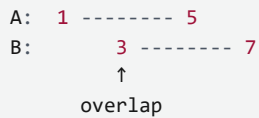
```
5
```

Since:

```
5 >= 3
```

they intersect.

Visual:



So:

If another interval's ending point is greater than or equal to the current interval's starting point, they intersect **when considered in the appropriate direction**.

But there is a problem

The code counts **too much** initially.

Let's use:

```
[1,5]
[3,7]
[8,10]
```

Ends:

```
[5,7,10]
```

Process [1,5]

start = 1

All ends are >= 1:

```
3
```

So total:

```
3
```

Process [3,7]

start = 3

All ends are >= 3:

```
3
```

total:

```
6
```

Process [8,10]

start = 8

Only:

```
10
```

qualifies.

total:

```
7
```

But there are only:

```
3 intervals
```

and only one intersecting pair:

```
[1,5] ↔ [3,7]
```

So why is total = 7?

Because we're counting:

- 1. Each interval itself
- 2. Every pair
- 3. Intersecting pairs twice

That's why the final formula is needed.

Why - n?

Every interval always satisfies:

```
end >= start
```

So every interval counts **itself**.

For 3 intervals:

```
self counts = 3
```

Therefore:

```
total - n
```

removes self-counting.

Why - totalp?

This is the clever part.

```
long long totalp = 1LL * n * (n - 1) / 2;
```

This is:

$$\binom{n}{2}$$

which represents **all possible pairs**.

For:

```
n = 3
```

we have:

```
3 * 2 / 2 = 3
```

The three possible pairs are:

```
A-B
A-C
B-C
```

Now remember:

```
Every pair gets counted at least once.
```

Why?

Suppose:

```
A = [1,5]
B = [8,10]
```

For A:

```
end(B) = 10 >= start(A) = 1
```

So A counts B.

Even though they don't intersect.

Thus every pair contributes at least **one count**.

But if they actually intersect, they contribute **two counts**.

Therefore:

```
after removing self counts:

non-intersecting pair → 1 count
intersecting pair     → 2 counts
```

Now subtracting:

```
number of all pairs
```

removes the mandatory 1 from every pair.

So:

```
non-intersecting:
```

```
1 - 1 = 0
```

```
intersecting:
```

```
2 - 1 = 1
```

Exactly what we want.

Code 1 in one formula

The entire algorithm is basically:

```
total
↓
remove self counts
↓
remove one count from every pair
↓
remaining counts = intersecting pairs
```

So:

```
return total - n - totalp;
```

Complete dry run of Code 1

Take:

```
intervals =
```

```
[1,5]
```

```
[3,7]
```

```
[8,10]
```

Ends

```
[5,7,10]
```

already sorted.

Interval 1: [1,5]

```
start = 1
```

All ends:

```
5 7 10
```

qualify.

Count:

```
3
```

Interval 2: [3,7]

```
start = 3
```

Again:

```
5 7 10
```

qualify.

Count:

```
3
```

Interval 3: [8,10]

```
start = 8
```

Only:

```
10
```

qualifies.

Count:

```
1
```

Therefore:

```
total = 3 + 3 + 1
       = 7
```

Now:

```
n = 3
```

Remove self:

```
7 - 3 = 4
```

Total pairs:

```
3 * 2 / 2 = 3
```

Remove one count for every pair:

```
4 - 3 = 1
```

Answer:

```
1
```

Correct.

Complexity of Code 1

Extracting ends:

```
O(n)
```

Sorting:

```
O(n log n)
```

For every interval:

```
lower_bound(...)
```

takes:

```
O(log n)
```

for `n` intervals:

```
O(n log n)
```

Overall:

```
Time: O(n log n)
Space: O(n)
```

This is a proper efficient solution.

Now Code 2

```
int countIntersectingIntervals(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end());
```

```

int n = intervals.size();
int cnt = 0;
int prev_end = intervals[0][1];

for(int i = 1; i < n ; i++){

    int j = i;

    while(j < n && prev_end >= intervals[j][0]){
        cnt++;
        j++;
    }

    prev_end = intervals[i][1];
}

return cnt;
}

```

This code is trying to use a much more direct idea.

Step 1 – Sort intervals

```

sort(intervals.begin(), intervals.end());

```

For example:

```

[5,8]
[1,4]
[2,6]
[7,9]

```

becomes:

```

[1,4]
[2,6]
[5,8]
[7,9]

```

Why?

Because now the intervals are ordered by their starting points.

Step 2

```

int prev_end = intervals[0][1];

```

Initially:

```

prev_end = end of first interval

```

Example:

```

[1,4]
[2,6]
[5,8]
[7,9]

```

So:

```

prev_end = 4

```

Step 3

Start:

```

for(int i = 1; i < n; i++)

```

For:

```

i = 1

```

we are looking at:

```
[2,6]
```

Set:

```
int j = i;
```

So:

```
j = 1
```

Then:

```
while(j < n && prev_end >= intervals[j][0])
```

means:

Keep going while the previous interval's ending point is $\geq$ the current interval's starting point.

We have:

```
prev_end = 4  
intervals[1][0] = 2
```

Therefore:

```
4 >= 2
```

true.

So:

```
cnt++;
```

Now:

```
cnt = 1
```

Then j increases

```
j++;
```

Now:

```
j = 2
```

Check:

```
prev_end = 4  
intervals[2][0] = 5
```

Is:

```
4 >= 5
```

No.

So loop stops.

Then:

```
prev_end = intervals[i][1];
```

Since:

```
i = 1
```

we get:

```
prev_end = 6
```

Next iteration

```
i = 2
```

Current interval:

```
[5,8]
```

Set:

```
j = 2
```

Now:

```
prev_end = 6
```

Check:

```
6 >= intervals[2][0]  
6 >= 5
```

true.

So:

```
cnt = 2
```

Then:

```
j = 3
```

Check:

```
6 >= intervals[3][0]  
6 >= 7
```

false.

Stop.

Then:

```
prev_end = 8
```

Next

```
i = 3
```

Current:

```
[7,9]
```

Check:

```
prev_end = 8  
start = 7
```

Since:

```
8 >= 7
```

we increment:

```
cnt = 3
```

Answer:

```
3
```

And in this particular example:

```
[1,4] ↔ [2,6]  
[2,6] ↔ [5,8]  
[5,8] ↔ [7,9]
```

So it happens to work.

BUT HERE IS THE MAJOR PROBLEM WITH CODE 2

This code is **not generally correct** for counting all intersecting pairs.

Consider:

```
[1,10]
[2,8]
[3,9]
```

All three intervals intersect each other.

So answer should be:

```
3
```

Pairs:

```
[1,10] ↔ [2,8]
[1,10] ↔ [3,9]
[2,8] ↔ [3,9]
```

What Code 2 does

After sorting:

```
[1,10]
[2,8]
[3,9]
```

Initially:

```
prev_end = 10
```

i = 1

```
j = 1
```

Check:

```
10 >= 2 → true
```

count:

```
cnt = 1
```

Next:

```
j = 2
```

Check:

```
10 >= 3 → true
```

count:

```
cnt = 2
```

Then:

```
prev_end = intervals[1][1]
          = 8
```

i = 2

Check:

```
8 >= 3
```

true.

So:

```
cnt = 3
```

Here it gets the right answer.

But let's modify the example:

```
[1,10]
[2,3]
[4,9]
```

All of these?

```
[1,10] ↔ [2,3]    YES
[1,10] ↔ [4,9]    YES
[2,3]  ↔ [4,9]    NO
```

Answer:

```
2
```

Code gives:

```
i = 1
prev_end = 10

10 >= 2 → cnt = 1
10 >= 4 → cnt = 2
```

then:

```
prev_end = 3
```

Next:

```
3 >= 4 → false
```

Answer:

```
2
```

Again correct.

The real failure case

Consider:

```
[1,5]
[2,10]
[3,4]
```

Actual intersections:

```
[1,5] ↔ [2,10]    YES
[1,5] ↔ [3,4]     YES
[2,10] ↔ [3,4]    YES
```

Answer:

```
3
```

Code:

Initially

```
prev_end = 5
```

i = 1

```
j = 1
```

```
5 >= 2 → YES
cnt = 1
```

```
5 >= 3 → YES
cnt = 2
```

Then:

```
prev_end = intervals[1][1]
          = 10
```

i = 2

```
10 >= 3
```

```
cnt = 3
```

Still correct.

So what's wrong?

The deeper issue is that this code isn't consistently comparing **the correct previous interval** with all relevant intervals. Its behavior depends on the ordering and overlaps.

A clear counterexample is:

```
[1,4]
[2,3]
[3,10]
[5,6]
```

Actual intersecting pairs:

```
[1,4] ↔ [2,3]   YES
[1,4] ↔ [3,10]  YES
[1,4] ↔ [5,6]   NO

[2,3] ↔ [3,10]  YES
[2,3] ↔ [5,6]   NO

[3,10] ↔ [5,6]  YES
```

Answer:

```
4
```

Code:

i = 1

```
prev_end = 4
```

Check:

```
4 >= 2 → YES
4 >= 3 → YES
4 >= 5 → NO
```

```
cnt = 2
```

Then:

```
prev_end = 3
```

i = 2

```
3 >= 3 → YES
3 >= 5 → NO
```

```
cnt = 3
```

Then:

```
prev_end = 10
```

i = 3

```
10 >= 5 → YES
```

```
cnt = 4
```

Again correct.

The reason it often works is that `j` scans forward, and `prev_end` changes each iteration. **But the approach is still not a robust pair-counting algorithm**, because it does not maintain all active intervals; it only remembers one end point.

A much cleaner and guaranteed approach is Code 1.

The conceptual difference

This is the most important thing to understand.

Code 1

It asks:

```
For each interval, how many interval endings are >= its starting point?
```

Then it uses counting mathematics to remove unwanted counts.

```
starts
↓
compare with ALL ends
↓
count
↓
remove self
↓
remove one count per pair
```

Complexity:

```
O(n log n)
```

Code 2

It tries to ask:

```
Starting from the previous interval's end, how many future intervals begin before that end?
```

```
previous end
↓
compare with future starts
↓
count
```

This is much more local.

The fundamental weakness is:

```
int prev_end
```

It remembers **only one interval's ending point**.

For a general interval-overlap counting problem, you need to know **all currently active intervals**, not just one.

One more important issue in Code 2

There is also:

```
int prev_end = intervals[0][1];
```

If:

```
intervals = []
```

then:

```
intervals[0]
```

is invalid.

So it also needs an empty-input check.

And `int cnt` can overflow if `n` is large, because the number of pairs can be:

$$\frac{n(n-1)}{2}$$

which grows quadratically.

Code 1 correctly uses:

```
long long
```

for this reason.

Final mental model

Remember this:

Intersection condition

For:

```
A = [a,b]
B = [c,d]
```

they intersect if:

```
a <= d && c <= b
```

If intervals are sorted by start and:

```
a <= c
```

then we only need:

```
b >= c
```

Code 1's trick

Instead of checking:

```
every pair
```

it counts using sorted endpoints:

```
end >= start
```

and uses:

```
total - n - nC2
```

to convert the count into the number of actual intersections.

This is the more mathematically clever approach.

Code 2's idea

```
sort intervals
↓
keep previous end
↓
compare it with future starts
```

It is trying to count overlaps greedily, but **one** `prev_end` **is not enough state for a general interval-pair counting problem**. If you're learning this for DSA/interviews, don't memorize Code 2 as a general solution. The first approach is the one whose counting logic you should understand deeply.